CS/CE 224/227: Object Oriented Programming and Design Methodologies

Week 05/Unit 05: Object Oriented Programming

Course taught by: Zubair Irshad

Slides Coutesy: Faisal Alvi

(For any suggested modifications please email: faisal.alvi@sse.habib.edu.pk)

Unit Outline

1. Struct In C++
2. Classes and Objects
3. Private and Public Class Members
4. Constructors
5. Initializer Lists
6. Summary.

Struct

- Recall from HW1, a **struct** in C++ groups together **a collection of variables** (called members), which can be of **different types**.
- The variables in a structure can be of different types: Some can be `int`, some can be `float`, and so on.

```
struct {              // Structure declaration
  int myNum;          // Member (int variable)
  string myString;    // Member (string variable)
} myStructure;        // Structure variable
```

```
int main() {
  myStructure.myNum = 1;                    // Assign value to integer member
  myStructure.myString = "Hello World!";    // Assign value to string member

  cout << myStructure.myNum << "\n";        // Print integer member
  cout << myStructure.myString << "\n";     // Print string member

  return 0;
}
```

Classes and Objects (Motivation)

- Imagine you're building a **game**. You want to keep track of multiple players. Each player has:
- A **name**
- A **score**
- A **health level**
- The ability to **jump**, **attack**, or **heal**
- If we try to manage all of this with **primitive types** (int, float, string, etc.), things get messy fast. You'd have:

Classes and Objects (Motivation)

```
string name1, name2, name3;
int score1, score2, score3;
float health1, health2, health3;
```

- This becomes error-prone, hard to manage, and not scalable.
- Imagine re-doing this for each player and there are **n** players where **n** is a large number

Classes and Objects (Motivation)

- A **class** allows you to **define your own data type**—just like how int is for integers, **you can define a Player class** that *contains* name, score, and health, and *does* jump, attack, heal. A major motivation here is **reusability.**

```
class Player {
public:
    string name;
    int score;
    float health;
    void jump() {
        // jumping logic
    }
    void attack() {
        // attack logic
    }
    void heal() {
        // heal logic
    }
};
```

```
int main() {

    Player player1, player2;
    player1.name = "Alice";
    player2.name = "Bob";

    player1.score = 0;
    player2.score = 0;
    player1.health = 100.0f;
    player2.health = 100.0f;

    return 0;
}
```

Classes and Objects (Motivation)

- **Analogy: Blueprint and House**
  - A **class** is like a **blueprint** for a house.
  - An **object** is like a **house built from the blueprint**.
  - You can build many houses (objects) from one blueprint (class), each with different colors, rooms, and furniture (data).

Classes and Objects (Formal Definition)

- A **class** in C++ is a user-defined data type that groups **data members** (variables) and **member functions** (methods) under one name.
- It serves as a definition of a **type**, specifying the structure and behavior of entities of that type.
- The class does not consume memory until an **object** is created.
- **An object is an instance of a class.**
- Let’s see another example:

Analysis of the Example

```
class smallobj
//define a class
{
    private:
        int somedata; //class data
    public:
        void setdata(int d) //member function to set data
        { somedata = d; }

        void showdata() //member function to display data
        { cout << "Data is " << somedata << endl; }
};
```

Class Definition

Private Variables

Public Functions

Analysis of the Example

```
int main()
{
    smallobj s1, s2; two objects of class smallobj
    s1.setdata(1066); //call member function to set data
    s2.setdata(1776);
    s1.showdata(); //call member function to display data
    s2.showdata();
    return 0;
}
```

Object Instantiation

Calling functions

Classes

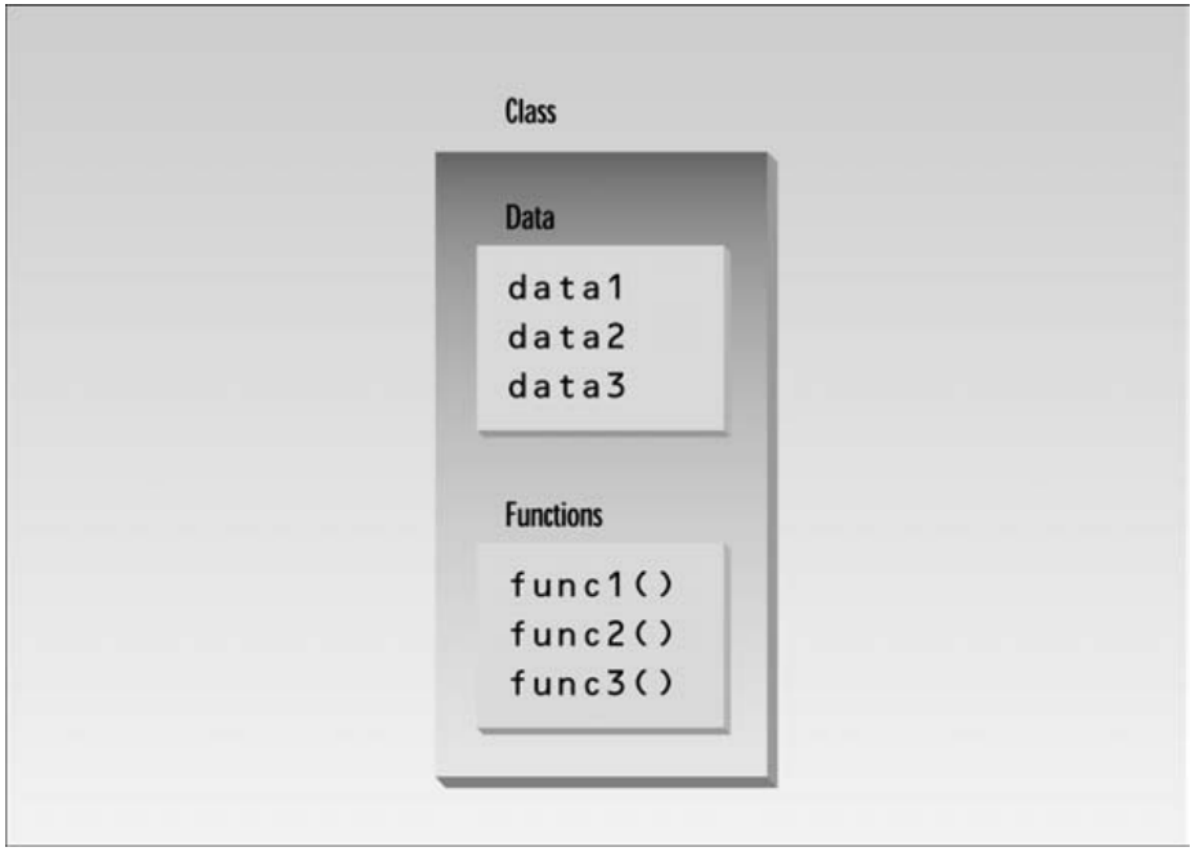


FIGURE 6.1
Classes contain data and functions.

Class Members

- **Members of a class** refer to the **variables and functions** that are **declared within the class definition**.
- These members define the data and behavior associated with objects of that class.
  - **Data Members:** These are **variables declared inside the class** that hold the **state (data)** of the objects.
  - **Member Functions:** These are **functions defined inside the class** that operate on the data members. They define the **behavior** of the class.

Private and Public Class Members

- A key feature of object-oriented programming is **data hiding**.
- It means that data is concealed within a class so that it cannot be accessed by functions outside the class.
- The primary mechanism for hiding data is to put it in a class and make it private.
- **Private data** or functions **can only be accessed from within the class.**
- **Public data or functions**, on the other hand, **are accessible from outside the class**.

Difference between Public and Private members

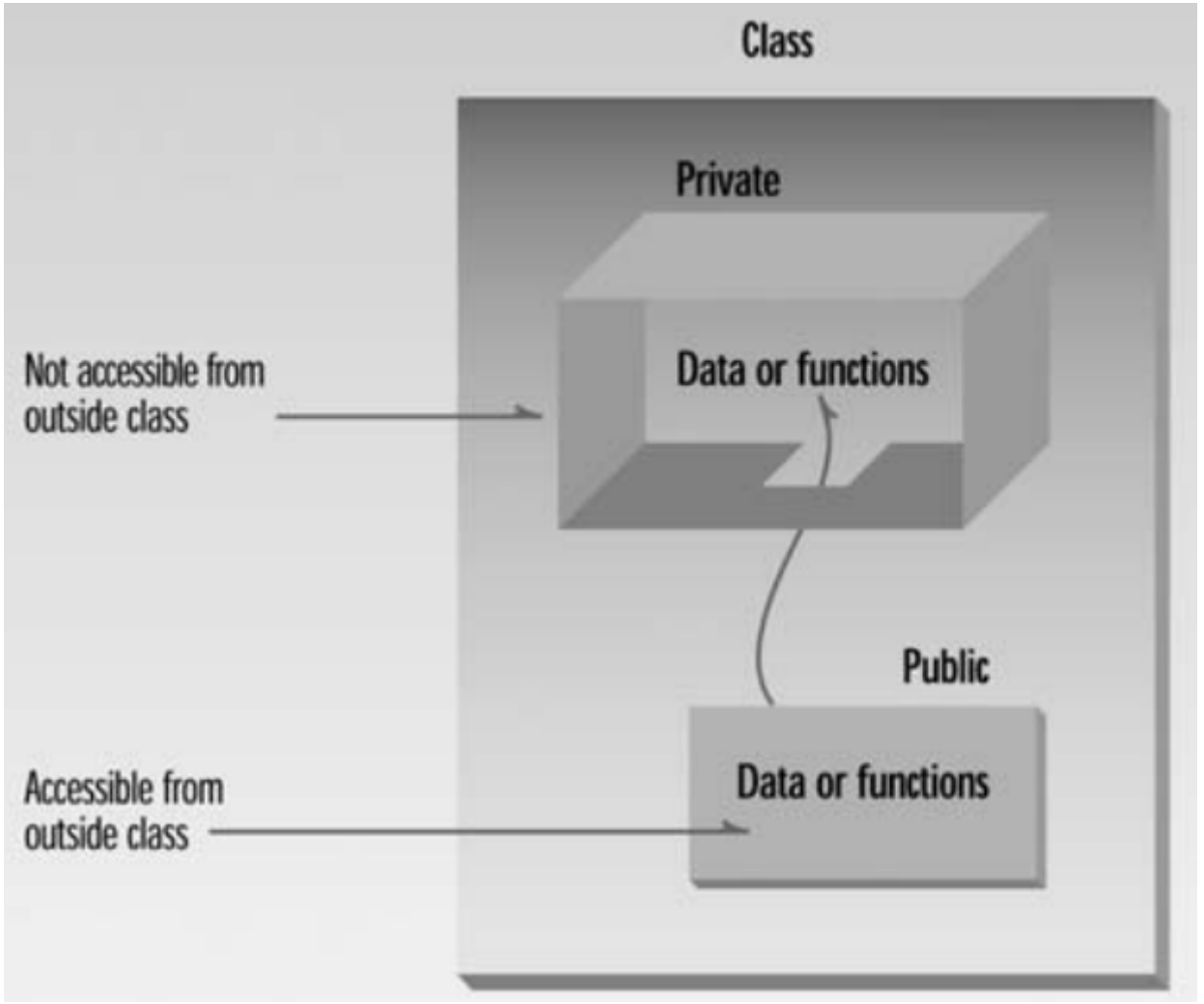
- Members declared as private **cannot be accessed** directly from outside the class. They are **only accessible** from within the class itself.

```cpp
#include <iostream>
using namespace std;

class MyClass {
    private:
        int secret = 42;

    public:
        void showSecret() {
            cout << "Secret is " << secret << endl;
        }
};

int main() {
    MyClass obj;
    // cout << obj.secret;    ❌ Error: 'secret' is private
    obj.showSecret();         // ✅ OK: uses public method to access private data
}
```

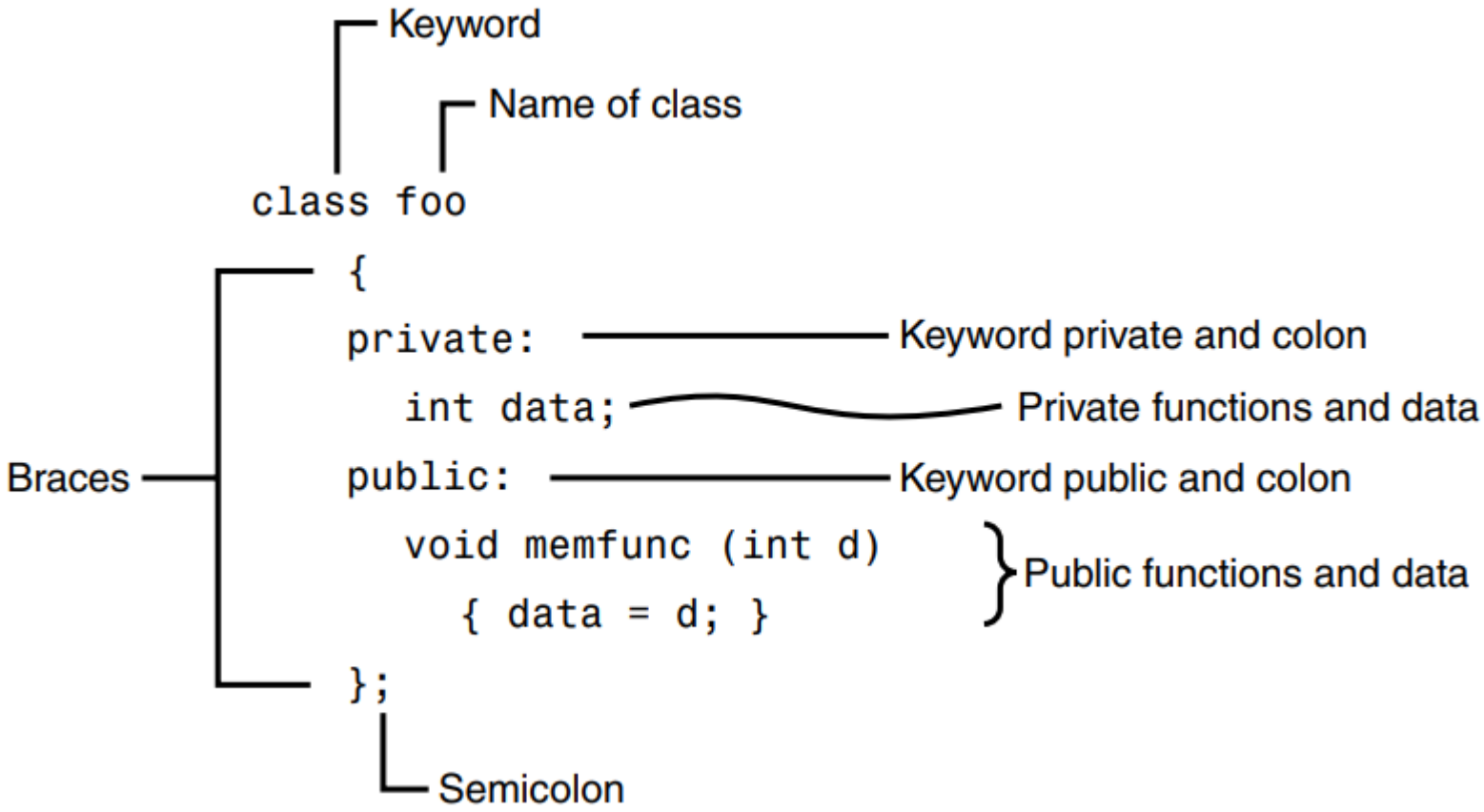
- Members declared as `public` **can be accessed freely** from anywhere in the program — including from outside the class.

```cpp
#include <iostream>
using namespace std;

class MyClass {
public:
    int id;

    void display() {
        cout << "ID: " << id << endl;
    }
};

int main() {
    MyClass obj;
    obj.id = 5;          // ✅ OK: public member
    obj.display();       // ✅ OK
}
```

Private and Public Class Members



Syntax of a Class Definition



Member Functions

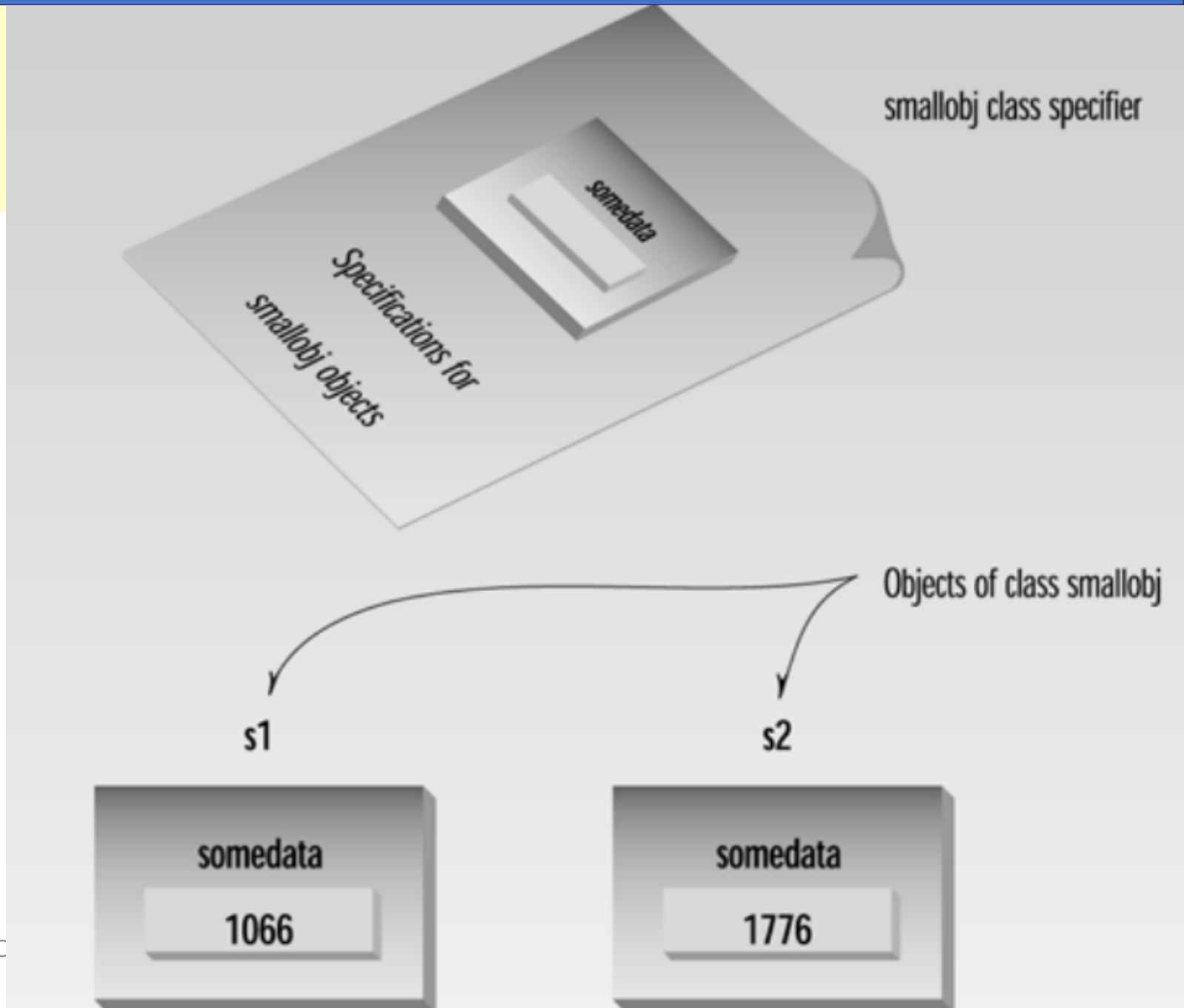
- The member functions in the smallobj class perform operations that are quite common in classes: **setting and retrieving the data** stored in the class.
- The setdata() function accepts a value as a parameter and sets the `somedata` variable to this value.
- The showdata() function displays the value stored in `somedata`.

Defining Objects

- The first statement in `main() smallobj s1, s2;` defines two objects, `s1` and `s2`, of class `smallobj`.
- Defining an object is similar to defining a variable of any data type: Space is set aside for it in memory.
- Defining objects in this way means creating them. This is also called **instantiating** them, because an instance of the class is created.
- An object is an **instance** (that is, a specific example) **of a class**.
- Objects are sometimes called **instance variables**.

Instantiating an Object

- Some object-oriented languages refer to calls to member functions as messages.
- Thus the call `s1.showdata();` can be thought of as sending a message to `s1` telling it to show its data.



Another Example

```
class part //define class
{
    private:
        int modelnumber; //ID number of widget
        int partnumber; //ID number of widget part
        float cost; //cost of part
    public:
        void setpart(int mn, int pn, float c) //set data
        {
            modelnumber = mn;
            partnumber = pn;
            cost = c;
        }

        void showpart() //display data
        {
            cout << "Model " << modelnumber;
            cout << ", part " << partnumber;
            cout << ", costs $" << cost << endl;
        }
};
```

Another Example

```
int main()
{
    part part1; //define object
// of class part
    part1.setpart(6244, 373, 217.55F); //call member function
    part1.showpart(); //call member function
    return 0;
}
```

Constructors

- In **C++**, a **constructor** is a special member function of a class that is **automatically called when an object of that class is created**. It is used to **initialize objects**.
- The term constructor is sometimes abbreviated `ctor`, especially in comments in program listings.
- Key features of constructors:
  - The constructor **has the same name as the class**.
  - It **does not have a return type**, not even void.
  - It can be **overloaded** (i.e., you can have multiple constructors with different parameters).
  - There is also a **default constructor** (no parameters) and a **parameterized constructor** (takes arguments).

Constructors

- Note that the name of the constructor is that of the class itself.
- Constructors do not have a return type.
- There are several ways to initialize variables using a constructor.
- Let's look at the example on the right to see one way to initialize a variable using a constructor!
- This was of **initialization** happens within the **body of constructor**
- Note how when you create instance of a class **mycar,** it automatically initializes *brand* and *year* values through **constructor**

```
class Car {
public:
    string brand;
    int year;

    // Constructor
    Car(string b, int y) {
        brand = b;
        year = y;
    }

    void displayInfo() {
        cout << "Brand: " << brand << ", Year: " << year << endl;
    }
};

int main() {
    // Creating an object and calling the constructor
    Car myCar("Toyota", 2020);
    myCar.displayInfo();

    return 0;
}
```

Constructor

Constructors: Initializer Lists

- Another way to initialize objects in class is through **constructor initializer list** in C++.
- It's a preferred way to initialize class member variables, especially when:
  - The member variables are const

```
MyClass() : member1(val1), member2(val2) {
    // constructor body (optional)
}
```

```
class Car {
private:
    const string brand; // const → must be initialized using initializer list
    int year;

public:
    // Constructor with initializer list
    Car(string b, int y) : brand(b), year(y) {
        // nothing else needed; already initialized above
    }

    void displayInfo() {
        cout << "Brand: " << brand << ", Year: " << year << endl;
    }
};

int main() {
    Car myCar("Toyota", 2020);
    myCar.displayInfo();
    return 0;
}
```

Summary

- This session was a first look at Object Oriented Programming.
- A class can be considered as an extension of a structure – contains both data and functions on that data.
- Access Modifiers such as public and private control access to both data and functions.
- A constructor is a special member function of a class that is automatically called when an object of that class is created. It is used to initialize objects.